SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

QUESTION 1: Online Banking Fund Transfer Failures

Answer:

1. Introduction

The online banking system experiences frequent failures during fund transfers, especially at minimum and maximum transfer limits. This indicates that the existing test cases do not adequately cover boundary values and input conditions. A structured black-box testing approach using Equivalence Partitioning (ECP) and Boundary Value Analysis (BVA) is required to identify defects effectively.

2. Analysis of the Existing Testing Strategy

The current testing strategy appears to depend heavily on random testing. Random testing selects inputs without systematically covering critical boundaries. Therefore, defects related to minimum and maximum transfer amounts may remain undetected.

For example, if the permitted transfer range is ₹100 to ₹1,00,000, testing only typical amounts such as ₹500 and ₹5,000 will not reveal errors at the limits.

3. Application of Equivalence Partitioning

Equivalence Partitioning divides the input domain into valid and invalid classes.

Assume the minimum transfer amount is ₹100 and the maximum is ₹1,00,000.

Equivalence Class	Test Input	Expected Result
Below minimum	₹99	Reject transfer
Minimum valid value	₹100	Accept transfer
Within valid range	₹5,000	Accept transfer
Maximum valid value	₹1,00,000	Accept transfer
Above maximum	₹1,00,001	Reject transfer
Zero amount	₹0	Reject transfer
Negative amount	−₹100	Reject transfer
Non-numeric input	ABC	Display validation error

4. Application of Boundary Value Analysis

Boundary Value Analysis focuses on values at and around the limits because errors frequently occur at these points.

Test Case	Transfer Amount	Expected Result
TC1	₹99	Reject
TC2	₹100	Accept
TC3	₹101	Accept
TC4	₹99,999	Accept
TC5	₹1,00,000	Accept

TC6	₹1,00,001	Reject

These tests verify whether the application correctly implements inclusive minimum and maximum limits.

5. Evaluation of Random Testing

Random testing may be retained as a supplementary technique because it can reveal unexpected defects. However, it should not be the primary testing strategy for financial boundary conditions.

A systematic approach provides predictable coverage and makes failures reproducible.

6. Proposed Optimized Testing Approach

1. Identify the minimum and maximum transfer limits.
2. Divide inputs into valid and invalid equivalence classes.
3. Apply Boundary Value Analysis around both limits.
4. Test insufficient account balance, daily transaction limits, invalid beneficiary details, and network failures.
5. Verify that failed transfers do not result in duplicate debits.
6. Perform regression testing after fixing defects.
7. Supplement systematic tests with random and exploratory testing.

7. Justification

ECP reduces the number of test cases by grouping similar inputs, while BVA targets high-risk boundary defects. Combining both techniques improves defect detection without requiring every possible amount to be tested. Additional transaction and failure-handling tests improve financial correctness and reliability.

Conclusion: A structured black-box testing strategy using ECP and BVA, supported by random testing, is more effective than relying on random testing alone. It ensures reliable transfer validation and reduces the risk of financial errors.

QUESTION 2: Healthcare Application – Insufficient Test Coverage

Answer:

1. Introduction

The healthcare application has experienced critical defects after deployment because its testing process did not provide sufficient coverage. Since healthcare software may influence patient safety, the testing strategy must examine program statements, decision branches, and critical execution paths.

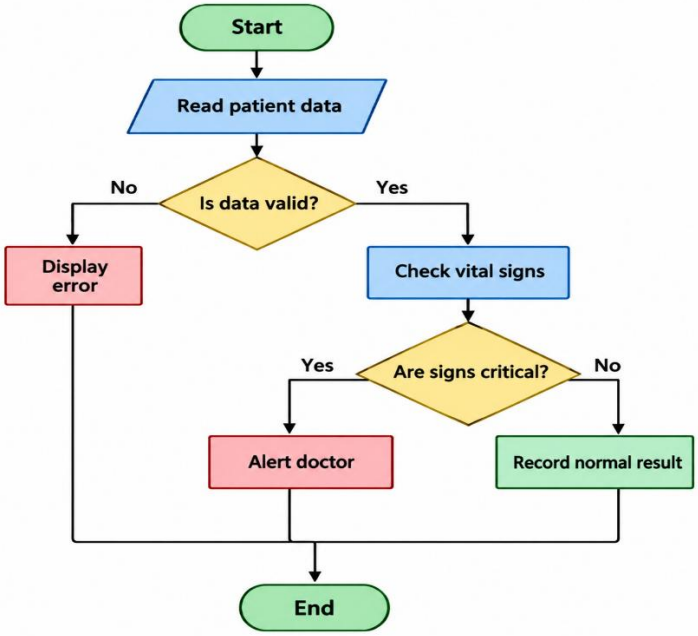
2. Analysis of Testing Gaps

The existing approach may have tested only normal input conditions without verifying all decision outcomes. Test adequacy criteria such as statement coverage, branch coverage, and path coverage may not have been properly evaluated.

For example, a patient-risk assessment module might calculate a result correctly for normal readings but fail when a reading is missing or exceeds a critical threshold.

3. Conceptual Control Flow Graph

Consider the following simplified patient-risk assessment logic:



The graph contains a start node, processing nodes, decision nodes, and an end node. Each decision creates alternative execution paths.

4. Missing Test Paths

Test Case	Input Condition	Expected Execution
TC1	Invalid or missing patient data	Display error
TC2	Valid data with critical vital signs	Alert doctor
TC3	Valid data with normal vital signs	Record normal result
TC4	Vital signs exactly at the critical threshold	Apply the specified threshold rule
TC5	Multiple invalid fields	Reject incomplete data safely

These cases cover the principal decision outcomes and important boundary conditions.

5. Evaluation of Coverage Criteria

- Statement coverage: Verifies that every executable statement runs at least once. It does not guarantee that every decision outcome is tested.
- Branch coverage: Verifies that each decision outcome, such as true and false, is executed.
- Path coverage: Examines sequences of control-flow decisions. Complete path coverage may be infeasible when loops or numerous independent decisions create many paths.

For example, a test that executes the error-handling statement may achieve statement coverage without testing the valid-data branch.

6. Improved Testing Strategy

1. Construct a control flow graph from the source code.
2. Identify every decision and its possible outcomes.

3. Design tests for valid, invalid, normal, and critical patient data.
4. Measure statement and branch coverage.
5. Identify independent paths and test all feasible critical paths.
6. Test missing values, invalid readings, threshold boundaries, and exception handling.
7. Perform integration testing with connected medical devices and databases.
8. Conduct regression testing after each correction.

7. Justification

Statement coverage confirms execution, while branch coverage provides stronger evidence that decisions have been tested. Path testing helps identify defects arising from particular sequences of decisions. Combining these criteria with boundary and exception tests increases confidence in the software.

Complete path coverage should not be claimed unless all feasible paths have actually been examined.

Conclusion: A coverage-driven white-box testing strategy using a control flow graph, branch analysis, and feasible path testing improves healthcare software reliability and reduces the risk of undetected defects.

QUESTION 3: E-Commerce System – Input Combination Errors

Answer:

1. Introduction

The e-commerce application calculates prices using price, quantity, and discount. Defects in specific input combinations indicate that the existing testing approach examines individual inputs without sufficiently testing their interactions. Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing should be combined.

2. Analysis of the Existing Approach

Testing price, quantity, and discount separately may not detect errors in the combined calculation.

For example, a discount may be applied twice, a negative quantity may be accepted, or the final amount may become negative. Therefore, the test strategy must validate both individual inputs and combinations of business rules.

3. Equivalence Partitioning

Assume the following illustrative business rules:

- Price must be greater than ₹0.
- Quantity must be a positive integer.
- Discount must be between 0% and 50%, inclusive.

Input	Valid Equivalence Class	Invalid Equivalence Class
Price	Positive monetary amount	Zero, negative, non-numeric
Quantity	Positive integer	Zero, negative, fractional, non-numeric
Discount	0%–50%	Below 0%, above 50%, non-numeric

These partitions reduce the number of tests while covering different categories of input errors.

4. Boundary Value Analysis

Variable	Test Values	Expected Result
Price	−₹1, ₹0, ₹1	Reject invalid prices; accept ₹1
Quantity	−1, 0, 1, 2	Reject non-positive quantities; accept positive integers
Discount	−1%, 0%, 1%, 49%, 50%, 51%	Accept only values from 0% through 50%

Additional tests should verify currency precision and maximum permitted purchase quantities if those limits are specified.

5. Decision Table Testing

A decision table represents combinations of conditions and their corresponding actions.

Condition or Action	Rule 1	Rule 2	Rule 3	Rule 4
Price is valid	Yes	Yes	No	Yes
Quantity is valid	Yes	Yes	Yes	No
Discount is valid	Yes	No	Yes	Yes
Expected action	Calculate total	Reject discount	Reject price	Reject quantity

6. Revised Test Design Strategy

1. Partition each input into valid and invalid classes.
2. Test the minimum and maximum permitted values.
3. Create a decision table for business-rule combinations.
4. Test valid and invalid combinations of price, quantity, and discount.
5. Verify the calculated total against an independently calculated expected value.
6. Check that invalid input does not produce a negative or otherwise incorrect total.
7. Use pairwise testing for larger combinations where exhaustive testing is impractical.
8. Apply regression testing to previously identified defects.

7. Justification

ECP identifies representative input categories, BVA detects limit-related defects, and decision tables reveal interaction errors. Pairwise testing can reduce redundant tests when many variables are involved, although it does not guarantee detection of every higher-order interaction.

Conclusion: Combining ECP, BVA, and decision table testing provides a structured and efficient approach to detecting input-validation and calculation defects in an e-commerce system.

QUESTION 4: Complex Nested Conditions and Loops – Partial Path Coverage

Answer:

1. Introduction

The software module contains nested conditions and loops, but the team has achieved only partial path coverage. This suggests that certain combinations of decisions, loop conditions, or exit paths have not been tested. A control flow graph and basis path testing approach can improve coverage.

2. Analysis of the Existing Approach

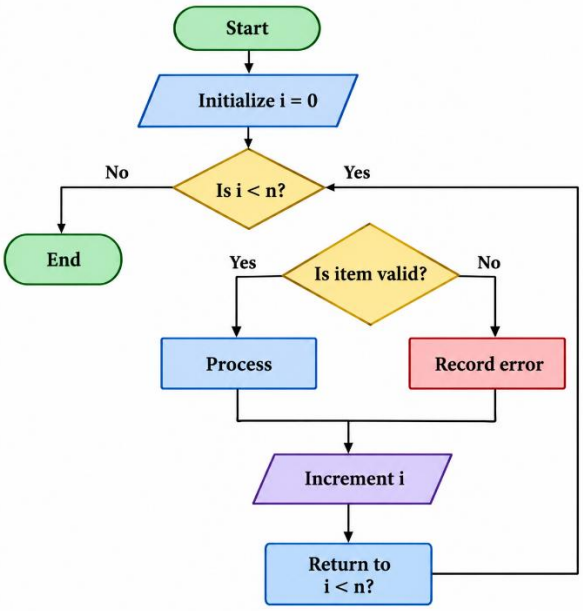
Partial path coverage may result from:

- Missing decision nodes in the control flow graph.
- Failure to identify independent execution paths.
- Testing only the normal loop execution.
- Ignoring zero iterations and boundary iteration counts.
- Overlooking exception and early-exit paths.

These omissions can allow logical errors and loop termination defects to remain undetected.

3. Illustrative Control Flow Graph

Consider the following pseudocode:



The graph represents the loop condition, an internal decision, and two alternative processing paths.

4. Identification of Independent Paths

The principal paths are:

1. The loop is skipped because \(n=0\).
2. The loop executes and the item is valid.
3. The loop executes and the item is invalid.
4. The loop executes several times with a mixture of valid and invalid items.

If additional nested conditions exist, their outcomes must also be represented in the graph.

5. Proposed Test Cases

Test Case	Input Condition	Expected Result
TC1	(n=0)	Loop body is skipped
TC2	(n=1), valid item	Item is processed
TC3	(n=1), invalid item	Error is recorded
TC4	Multiple valid items	All items are processed
TC5	Mixed valid and invalid items	Processing and error handling are correct
TC6	Maximum supported input size	Loop terminates correctly
TC7	Boundary values for nested conditions	Correct branches are executed

6. Cyclomatic Complexity and Coverage

Cyclomatic complexity estimates the number of linearly independent paths in a control flow graph.

For a connected graph with one entry point:

[V(G)=E-N+2]

where:

- (E) = number of control-flow edges.
- (N) = number of nodes.

Alternatively, for a conventional single-entry, single-exit graph with binary decision nodes:

[V(G)=D+1]

where (D) is the number of decision nodes.

The actual complexity must be calculated from the program's real control flow graph. A set of basis paths should then be selected to exercise independent control-flow behaviour.

7. Improved Testing Strategy

1. Construct the complete control flow graph.
2. Calculate cyclomatic complexity.
3. Identify a basis set of independent paths.
4. Design at least one test for each selected basis path.
5. Test zero, one, and multiple loop iterations.
6. Verify loop termination, early exits, and exception handling.
7. Measure branch coverage and inspect untested paths.
8. Use additional tests for safety-critical combinations and loop boundaries.

8. Justification

Basis path testing systematically covers independent paths and exposes logical defects that normal-case testing may miss. Loop-boundary testing is essential because initialization, termination, and iteration errors often occur at zero, one, or maximum iterations.

Conclusion: A complete control flow graph, cyclomatic complexity analysis, basis path testing, and loop-boundary testing provide a structured method for improving defect detection in modules with nested conditions and loops.

QUESTION 5: Financial Application – Hybrid Black-Box and White-Box Testing

Answer:

1. Introduction

The financial application combines black-box and white-box testing, yet defects continue to appear in production. This indicates that using both approaches alone does not guarantee adequate coverage. Their effectiveness depends on correct test design, measurable adequacy criteria, and systematic defect analysis.

2. Critical Analysis of the Existing Strategy

Possible weaknesses include:

- Insufficient testing at monetary boundaries.
- Incomplete validation of user inputs and business rules.
- Low branch coverage in internal calculations.
- Untested independent execution paths.
- Inadequate testing of transaction failures and exception handling.
- Failure to repeat regression tests after modifications.

A high number of test cases does not necessarily indicate good coverage if important conditions remain untested.

3. Black-Box Testing Strategy

Equivalence Partitioning divides monetary inputs into valid and invalid categories. Boundary Value Analysis tests values around the minimum and maximum transaction limits.

For an illustrative transfer range of ₹100 to ₹1,00,000:

Test Input	Technique	Expected Result
₹99	BVA	Reject
₹100	BVA	Accept
₹101	BVA	Accept
₹50,000	ECP	Accept
₹1,00,000	BVA	Accept
₹1,00,001	BVA	Reject
Negative amount	ECP	Reject

4. White-Box Testing Strategy

White-box testing examines the internal control structure of the application.

The team should:

1. Construct a control flow graph for the transfer process.
2. Identify decisions for authentication, balance validation, transaction limits, and transfer confirmation.
3. Test true and false outcomes of each decision.
4. Identify independent paths and perform basis path testing.
5. Test exception handling, rollback, and failure recovery.

For example, tests should cover successful transfer, insufficient balance, failed authentication, limit violation, and transaction-processing failure.

5. Test Adequacy Criteria

Criterion	Purpose
Statement coverage	Checks whether executable statements run
Branch coverage	Checks whether decision outcomes run
Path testing	Examines selected independent execution paths
Boundary coverage	Verifies behavior at input limits
Requirement coverage	Confirms that specified business rules are tested
Regression testing	Checks that fixes have not introduced new defects

Coverage measurements should be interpreted together. Statement coverage alone cannot demonstrate that all decision outcomes have been tested.

6. Proposed Hybrid Testing Strategy

1. Convert financial requirements into testable conditions.
2. Apply ECP and BVA to transaction inputs.
3. Use decision tables for combinations of account status, balance, and transaction limits.
4. Construct a control flow graph for transaction processing.
5. Perform branch and basis path testing.
6. Test duplicate requests, concurrent transfers, network interruptions, and rollback behavior.
7. Automate repeatable functional and regression tests.
8. Review defects and update the test suite whenever a new failure is discovered.

7. Justification

Black-box testing checks whether the application behaves according to requirements, while white-box testing examines the internal implementation. Their combination detects both externally visible failures and internal logic defects.

Additional transaction-integrity tests are essential because coverage percentages alone cannot guarantee the correctness of financial operations.

Conclusion: An integrated testing strategy combining ECP, BVA, decision tables, control flow graphs, branch coverage, basis path testing, and regression testing offers stronger defect prevention than an unstructured combination of testing methods.

QUESTION 6: HR Payroll System – Incorrect Bonus at 5, 10, and 15 Years

Answer:

1. Introduction

The HR payroll system calculates employee bonuses based on years of service and performance ratings. Employees with exactly 5, 10, and 15 years of service receive incorrect payouts. This suggests that boundary conditions, comparison operators, or bonus-category transitions may have been implemented incorrectly.

2. Analysis of the Failure

The errors may arise from incorrect use of operators such as <, <=, >, and >=. For example, if the bonus category changes at five years, a condition that incorrectly excludes exactly five years may assign the employee to the wrong category.

The actual bonus thresholds and payout rules must be obtained from the payroll requirements before expected monetary amounts can be finalized.

3. Application of Equivalence Partitioning

The service duration can be divided into equivalence classes based on the specified bonus policy.

Equivalence Class	Example Input	Expected Behavior
Below 5 years	4 years	Apply the applicable lower-tier rule
Exactly 5 years	5 years	Apply the specified 5-year rule
Between 5 and 10 years	7 years	Apply the applicable middle-tier rule
Exactly 10 years	10 years	Apply the specified 10-year rule
Between 10 and 15 years	12 years	Apply the applicable higher-tier rule
Exactly 15 years	15 years	Apply the specified 15-year rule
Above 15 years	16 years	Apply the applicable upper-tier rule

These classes assume the stated years are meaningful thresholds in the bonus policy.

4. Application of Boundary Value Analysis

Test Case	Service Duration	Purpose
TC1	4 years 11 months	Immediately below 5 years
TC2	Exactly 5 years	Verify the 5-year threshold
TC3	5 years 1 month	Immediately above 5 years
TC4	9 years 11 months	Immediately below 10 years
TC5	Exactly 10 years	Verify the 10-year threshold
TC6	10 years 1 month	Immediately above 10 years

TC7	14 years 11 months	Immediately below 15 years
TC8	Exactly 15 years	Verify the 15-year threshold
TC9	15 years 1 month	Immediately above 15 years

If the policy counts only completed years, the test data should instead follow that definition and test adjacent completed-year values.

5. Testing Performance Ratings

Performance ratings must also be tested independently and in combination with service duration.

Service Duration	Performance Rating	Expected Result
5 years	Lowest permitted rating	Apply the specified bonus rule
5 years	Highest permitted rating	Apply the specified bonus rule
10 years	Intermediate rating	Apply the specified bonus rule
15 years	Highest permitted rating	Apply the specified bonus rule
Valid service duration	Invalid rating	Reject or flag invalid input

6. Comparison with Exploratory Testing

Exploratory testing is useful for discovering unexpected behavior, but it does not guarantee systematic coverage of every threshold. BVA explicitly examines values immediately below, at, and above each boundary. ECP ensures that different categories of service duration and performance ratings are represented.

7. Improved Testing Strategy

1. Confirm the official bonus calculation rules.
2. Identify all service-duration and rating boundaries.
3. Apply ECP and BVA.
4. Verify the exact expected bonus against an independently calculated result.
5. Test rounding, missing ratings, and invalid service records.
6. Perform regression testing after correcting the calculation.

Conclusion: ECP and BVA provide a systematic method for identifying payroll defects at 5-, 10-, and 15-year thresholds. Exploratory testing should complement, rather than replace, these techniques because payroll accuracy requires repeatable and measurable verification.

QUESTION 7: Laboratory Temperature Controller – MC/DC Testing

Answer:

1. Introduction

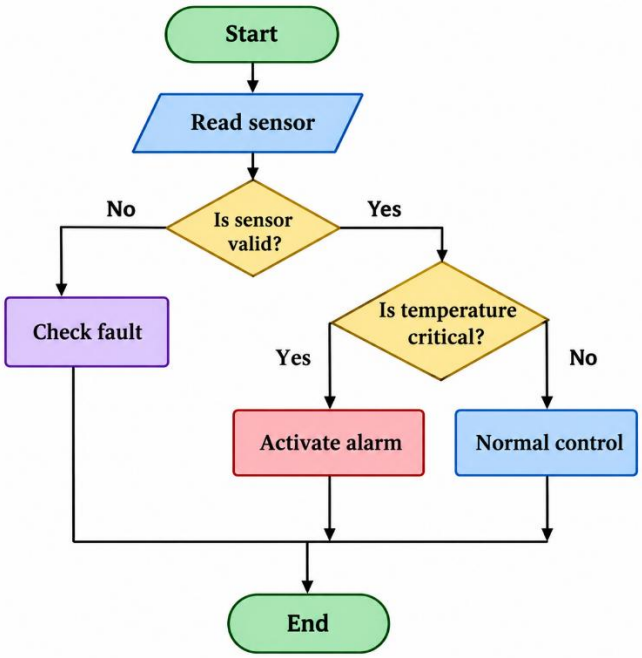
The laboratory temperature controller uses complex conditional logic to operate heaters, coolers, and alarms. The failure to activate an alarm during a sensor malfunction indicates that certain decision combinations may not have been tested. Since this is a safety-critical system, Modified Condition/Decision Coverage (MC/DC) is an appropriate testing criterion.

2. Illustrative Control Logic

Assume the controller uses these conditions:

- (A): Sensor reading is valid.
- (B): Temperature exceeds the critical limit.
- (C): Sensor malfunction is detected.

3. Conceptual Control Flow Graph



The graph shows the major control decisions. An actual graph should include all implemented conditions and the specific alarm-handling branches.

4. Limitations of Branch Coverage

Branch coverage checks whether each decision outcome has been executed. However, it does not necessarily demonstrate that each individual condition independently affects the overall decision.

For example, testing only normal temperatures and sensor failures might execute the alarm branch without proving that the critical-temperature condition independently controls the alarm output.

5. MC/DC Test Cases

For the illustrative expression \(D=(A\land B)\lor C\), the following table demonstrates how each condition can independently affect the decision outcome.

Test	A: Sensor Valid	B: Critical Temperature	C: Sensor Fault	Alarm D
T1	True	True	False	True
T2	True	False	False	False
T3	False	True	False	False
T4	False	False	True	True

The independent effects are demonstrated as follows:

- Condition A: Compare T1 and T3. Changing A changes the decision while B and C remain fixed.
- Condition B: Compare T1 and T2. Changing B changes the decision while A and C remain fixed.
- Condition C: Compare T2 and T4. Changing C changes the decision while A and B remain fixed.

This satisfies MC/DC for the stated Boolean expression, although the tests for a real controller must reflect its actual decision logic.

6. Proposed White-Box Testing Strategy

1. Construct a complete control flow graph.
2. Identify every Boolean condition in the alarm and control decisions.
3. Create MC/DC test pairs for each condition.
4. Test sensor disconnection, invalid readings, extreme temperatures, and recovery.
5. Verify that alarms activate within the required time.
6. Test the interaction between heaters, coolers, and alarms.
7. Measure MC/DC coverage using suitable coverage tools.
8. Repeat testing after every safety-related code change.

7. Justification

MC/DC verifies that each condition can independently influence its decision. This is stronger than statement coverage and provides more detailed evidence than branch coverage alone. It is particularly useful when complex Boolean expressions control safety-related behavior.

However, achieving MC/DC does not by itself prove complete system safety; hardware faults, timing, integration, and failure recovery must also be tested.

Conclusion: A control flow graph combined with MC/DC testing provides a systematic way to detect missing logical combinations and improve the reliability of laboratory temperature control and alarm functions.

QUESTION 8: Travel Booking Portal – Negative Total Amount from Promo Codes

Answer:

1. Introduction

The travel booking portal allows multiple promotional codes to be applied to a booking. Some combinations produce a negative total amount, indicating that the application does not correctly validate interactions between price, promotional rules, and user eligibility.

Testing each input independently is insufficient because the defect arises from a combination of conditions.

2. Analysis of Existing Testing Limitations

Testing a single promo code on a valid booking may produce the correct total, while applying two codes together may cause discounts to exceed the booking price.

Potential defects include:

- Applying mutually exclusive promotions together.

- Applying a fixed discount more than once.
- Applying discounts to an incorrect subtotal.
- Ignoring user-tier restrictions.
- Failing to enforce a minimum payable amount of zero.

3. Application of Decision Table Testing

Assume the following illustrative rules:

- Booking price must be positive.
- Only eligible promotions can be applied.
- Certain promotions cannot be combined.
- The final payable amount cannot be negative.

Condition or Action	Rule 1	Rule 2	Rule 3	Rule 4
Valid booking price	Yes	Yes	Yes	No
Promo code eligible	Yes	Yes	No	Yes
Promo combination permitted	Yes	No	Yes	Yes
Expected action	Calculate total	Reject combination	Reject promotion	Reject booking price

The complete decision table should be expanded to cover the actual promo types, stacking rules, and user-tier restrictions.

4. Illustrative Test Cases

Assume a booking price of ₹1,000 and two fixed discounts of ₹700 each. These values are examples, not actual portal rules.

Test Case	Booking Price	Promotion Combination	Expected Result
TC1	₹1,000	No promotion	Total ₹1,000
TC2	₹1,000	One eligible ₹200 discount	Total ₹800
TC3	₹1,000	Two permitted discounts totalling ₹300	Total ₹700
TC4	₹1,000	Two mutually exclusive promotions	Reject the combination
TC5	₹1,000	Discounts exceed the booking price	Apply the defined validation or cap rule; never return a negative total
TC6	₹0	Any promotion	Reject invalid booking price

TC7	₹1,000	Promotion not valid for the user's tier	Reject the promotion

5. Cause-Effect Graphing

Cause-effect graphing identifies input conditions and the outcomes they produce.

The causes include:

- Valid booking price.
- Valid promo code.
- Eligible user tier.
- Compatible promotion combination.

The effects include:

- Accept booking.
- Apply permitted discount.
- Reject an invalid promotion.
- Display a validation message.

Logical relationships between these causes and effects can be converted into a decision table and then into test cases.

6. Optimized Test Design

1. List all promotion types and eligibility rules.
2. Identify which promotions can be combined.
3. Create equivalence classes for booking prices and promo validity.
4. Use a decision table to represent important combinations.
5. Test discount totals below, equal to, and above the booking price.
6. Verify the final total independently.
7. Test multiple user tiers and promotion restrictions.
8. Use pairwise testing where appropriate to reduce redundant combinations, while retaining specific high-risk combination tests.

7. Justification

Decision table testing is effective when outputs depend on combinations of business conditions. Cause-effect graphing makes the relationships between inputs and outcomes explicit. Boundary tests ensure that the payable amount does not become negative or violate the stated financial rules.

Conclusion: Decision table testing and cause-effect graphing can systematically identify invalid promotional combinations. Combining these methods with boundary testing improves discount correctness while minimizing redundant test cases.

QUESTION 9: Legacy Database Migration Tool – Infinite Loop Crashes

Answer:

1. Introduction

The database migration tool achieves high statement coverage, yet it frequently crashes because of infinite loops. This demonstrates that executing every statement once does not guarantee that loops terminate correctly or that all control-flow paths have been tested.

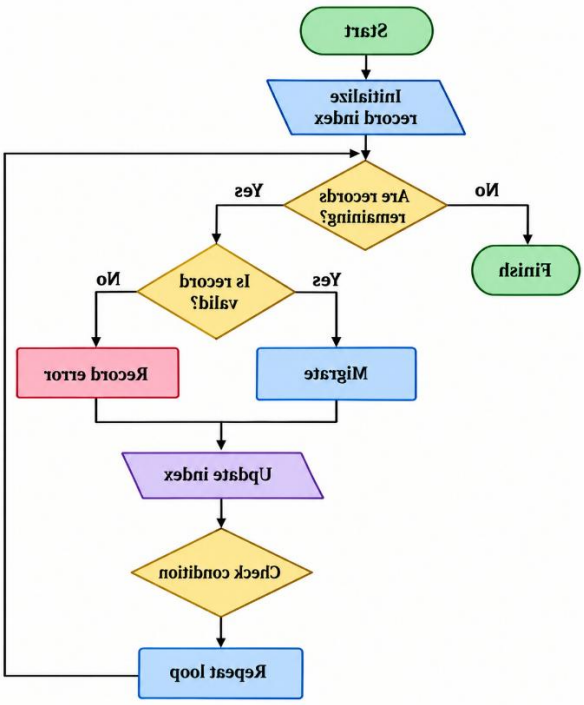
2. Why Statement Coverage Is Insufficient

Statement coverage confirms that executable statements have run, but it does not verify:

- Whether loop conditions eventually become false.
- Whether loop counters are updated correctly.
- Whether nested loops terminate.
- Whether empty datasets are handled correctly.
- Whether malformed records cause repeated processing.
- Whether early exits and error-handling paths behave correctly.

A loop body can execute and achieve statement coverage even if the loop never terminates for certain inputs.

3. Illustrative Control Flow Graph



The graph represents the record-processing loop and its termination condition.

4. Basis Path Testing

Basis path testing identifies a set of linearly independent paths through the control flow graph.

Representative paths include:

1. No records are available, so the loop is skipped.
2. A valid record is processed and the loop terminates.

3. An invalid record is rejected and processing continues.
4. A mixture of valid and invalid records is processed.
5. An exception occurs during migration and error handling is executed.

Additional independent paths must be identified if the actual program contains more decisions.

5. Loop Boundary Test Cases

Test Case	Input	Expected Result
TC1	Empty dataset	Terminate without processing records
TC2	One valid record	Migrate one record and terminate
TC3	One invalid record	Record the error and terminate
TC4	Multiple valid records	Process all records
TC5	Mixed valid and invalid records	Process or reject each record according to policy
TC6	Maximum supported dataset	Complete within acceptable limits
TC7	Malformed or repeated record	Avoid unintended repeated processing
TC8	Simulated processing exception	Recover or fail safely without hanging

6. Cyclomatic Complexity

Cyclomatic complexity estimates the number of independent paths in a control flow graph.

For a single connected component:

[V(G)=E-N+2]

where (E) is the number of edges and \(N\) is the number of nodes.

For a conventional graph with binary decisions, the same quantity can be calculated as:

[V(G)=D+1]

where (D) is the number of decision nodes.

The measured value helps determine a basis set of paths to test. It does not directly prove loop termination or identify every possible infinite-loop defect.

7. Improved Testing Strategy

1. Build the control flow graph from the source code.
2. Calculate cyclomatic complexity.
3. Select a basis set of independent paths.
4. Test zero, one, multiple, and maximum loop iterations.
5. Verify that loop counters and termination conditions are updated correctly.
6. Test malformed records, exceptions, and repeated records.
7. Use execution timeouts to detect potential hangs.
8. Perform regression testing after fixing loop defects.

8. Justification

Path testing examines different control-flow sequences, while loop-boundary testing specifically targets initialization, iteration, and termination defects. Execution timeouts provide an additional way to identify hangs that statement coverage cannot detect.

Conclusion: High statement coverage alone is insufficient for database migration software. Basis path testing, loop-boundary testing, cyclomatic complexity analysis, and termination checks provide stronger evidence of reliable record processing.

QUESTION 10: Food Delivery App – State Transition Testing

Answer:

1. Introduction

The mobile food delivery application uses the states Ordered, Preparing, Out for Delivery, and Delivered. Users have reported that the tracker becomes stuck when a driver cancels during delivery. This indicates that the current testing strategy does not adequately cover state transitions and exceptional events.

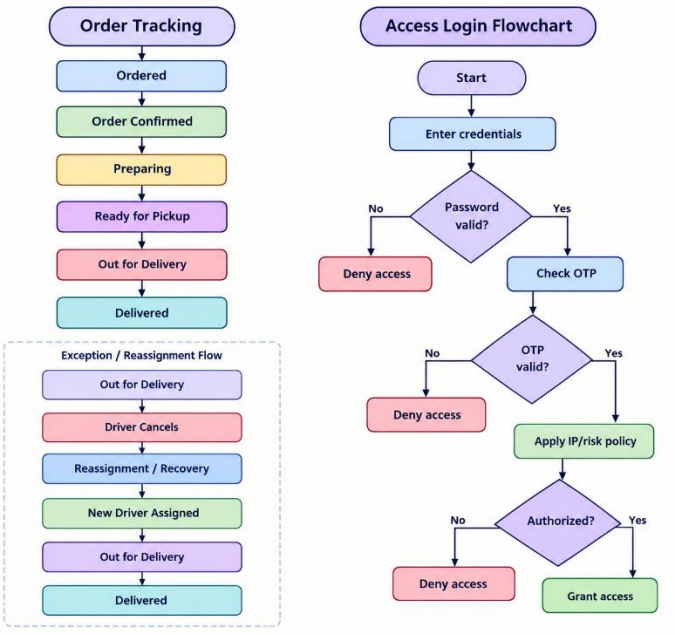
2. Analysis of the Existing Black-Box Testing Strategy

Equivalence Partitioning and Boundary Value Analysis are useful for input validation, but they are not sufficient for testing dynamic workflows. A food delivery tracker must correctly respond to events such as order confirmation, food preparation, driver assignment, driver cancellation, reassignment, and delivery completion.

The defect may occur because the application does not define a valid transition when a driver cancels after collecting or beginning to deliver an order.

3. State Transition Model

The principal states and transitions can be represented as follows:



If reassignment is impossible, the system should transition to a defined cancellation or support-resolution state according to the business rules. It must not remain indefinitely in an undefined state.

4. State Transition Test Cases

Test Case	Initial State	Event	Expected State
TC1	Ordered	Order confirmed	Preparing or the specified confirmed state
TC2	Preparing	Food ready and driver assigned	Ready for Pickup
TC3	Ready for Pickup	Driver collects order	Out for Delivery
TC4	Out for Delivery	Driver cancels	Reassignment or defined recovery state
TC5	Recovery state	New driver assigned	Out for Delivery
TC6	Out for Delivery	Delivery confirmed	Delivered
TC7	Delivered	Duplicate delivery event	Remain Delivered
TC8	Delivered	Attempt to return to Preparing	Reject invalid transition

The exact intermediate states must match the application's requirements.

5. State Coverage and Transition Coverage

- State coverage: Ensures that every defined state is visited.
- Transition coverage: Ensures that every defined transition is exercised.
- Transition-pair coverage: Tests sequences of two consecutive transitions to identify interaction defects.

For example, visiting the Out for Delivery state does not prove that the driver-cancellation transition has been tested.

6. Improved Testing Strategy

1. Identify all states and events.
2. Construct a state transition diagram or table.
3. Define valid and invalid transitions.
4. Test driver cancellation at different stages of delivery.
5. Verify reassignment, notifications, and status synchronization.
6. Test duplicate and delayed events.
7. Test network disconnection and application restart during transitions.
8. Verify that customers do not see contradictory delivery statuses.
9. Perform regression testing after fixing the transition logic.

7. Justification

State Transition Testing is particularly suitable for systems in which behavior depends on the current state and an event. It directly addresses workflow defects that may not be detected by testing individual inputs alone.

Conclusion: A state-based testing approach covering normal, exceptional, and invalid transitions can identify the cause of the stuck tracker and improve the reliability of the food delivery workflow.

QUESTION 11: Cybersecurity Authentication – Brute-Force Vulnerabilities

Answer:

1. Introduction

The cybersecurity authentication module validates password strength, OTP timing, and IP location. Despite testing, brute-force vulnerabilities have been discovered. This indicates that the testing process may not adequately examine repeated authentication attempts, internal control-flow paths, and combinations of security conditions.

2. Critical Analysis of the Existing Approach

Possible weaknesses include:

- Insufficient testing of repeated failed login attempts.
- Incomplete OTP expiry and retry-limit validation.
- Failure to enforce account or IP-based rate limits.
- Incorrect handling of authentication failures.
- Missing tests for account lockout and recovery.
- Inadequate analysis of the paths executed during authentication.

Testing a valid password and a valid OTP demonstrates only a normal authentication scenario; it does not establish resistance to brute-force attacks.

3. Application of Error Guessing

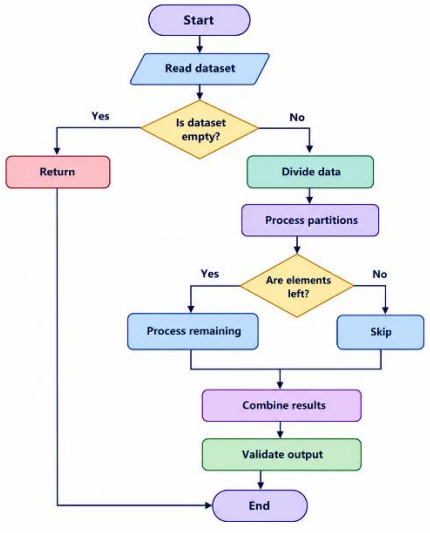
Error Guessing uses experience and knowledge of common failures to design targeted tests.

Test Scenario	Expected Secure Behavior
Repeated incorrect passwords	Apply configured rate limits or lockout controls
Expired OTP	Reject authentication
Repeated incorrect OTPs	Enforce retry limits
Correct password but incorrect OTP	Deny access
Unusual IP location	Apply the configured risk policy
Repeated login requests	Prevent unrestricted attempts
Password reset followed by login	Follow the specified session and authentication rules

These tests should be conducted only in an authorized test environment.

4. Formal Path Analysis

A simplified authentication flow is:



5. Hybrid Testing Strategy

1. Apply ECP to valid and invalid passwords, OTPs, and IP-risk categories.
2. Use BVA for OTP expiration times, retry limits, and rate-limit thresholds.
3. Apply error guessing to repeated attempts, session reuse, and recovery workflows.
4. Construct a control flow graph for the authentication logic.
5. Test true and false outcomes of authentication decisions.
6. Identify independent paths using cyclomatic complexity.
7. Verify rate limiting, lockout, logging, and alert generation.
8. Test concurrent requests and session invalidation.
9. Retest after security fixes and review relevant logs.

6. Justification

Error Guessing targets likely security weaknesses based on experience, while formal path analysis systematically examines internal decision sequences. Black-box testing verifies externally observable security behavior, and white-box testing helps locate missing checks in the implementation.

Neither method alone guarantees security. Authentication testing must also evaluate security requirements, configuration, and abuse-resistant controls.

Conclusion: A hybrid approach combining ECP, BVA, error guessing, and path analysis improves the detection of brute-force weaknesses and strengthens authentication reliability.

QUESTION 12: E-Learning Platform – Subscription Expiration and Upgrade Errors

Answer:

1. Introduction

The e-learning platform offers Free, Basic, and Premium subscriptions. Errors occur when users upgrade exactly on the expiration date. This suggests that the application may incorrectly classify date ranges or mishandle the transition between subscription states.

2. Analysis of Existing Testing Criteria

The testing process may have focused on upgrades well before or after expiration without checking the exact boundary. Date comparisons can produce errors when the implementation uses incorrect inclusive or exclusive operators.

Other possible causes include time-zone differences, incorrect renewal timestamps, and inconsistent subscription-type rules.

3. Equivalence Partitioning for Subscription Dates

Assume a subscription expires at a specified date and time. The expected behavior must be based on the documented expiration policy.

Equivalence Class	Test Condition	Expected Result
Before expiration	Upgrade before the expiry timestamp	Apply the valid upgrade policy
Exactly at expiration	Upgrade at the expiry timestamp	Apply the documented boundary rule
After expiration	Upgrade after the expiry timestamp	Apply the expired-subscription policy
Invalid date	Malformed date input	Reject invalid input
Missing expiration	Expiration date unavailable	Apply the specified error-handling rule

4. Equivalence Partitioning for Subscription Types

Current Subscription	Requested Subscription	Expected Behavior
Free	Basic	Apply the upgrade rules
Free	Premium	Apply the upgrade rules
Basic	Premium	Apply the upgrade rules
Premium	Basic	Apply downgrade rules, if supported
Expired Basic	Premium	Apply the expired-account policy
Invalid subscription type	Any upgrade	Reject invalid input

5. Boundary Value Analysis for Dates

Test Case	Time Relative to Expiration	Purpose
TC1	Just before expiration	Verify active subscription behavior
TC2	Exactly at expiration	Verify boundary interpretation
TC3	Just after expiration	Verify expired subscription behavior

TC4	Last second of the permitted period	Check final valid instant
TC5	First second after the permitted period	Check the first invalid instant

These tests should use the application's documented time zone and timestamp precision.

6. Improved Test Design

1. Document whether expiration is inclusive or exclusive.
2. Divide subscription dates into valid and invalid equivalence classes.
3. Test Free, Basic, and Premium subscription combinations.
4. Apply BVA around expiration timestamps.
5. Verify the resulting access permissions and billing records.
6. Test duplicate upgrade requests and payment failures.
7. Verify time-zone handling and date calculations.
8. Perform regression testing after correcting the transition logic.

7. Justification

ECP reduces redundant date and subscription tests by selecting representative classes. BVA focuses on the exact moment when subscription status changes. Combining them detects errors that ordinary upgrade testing may miss.

Conclusion: A structured date-based test suite using ECP and BVA improves subscription transition accuracy, prevents unintended access restrictions, and supports reliable billing and user access.

QUESTION 13: Smart-Home Lighting Controller – Branch and Condition Coverage

Answer:

1. Introduction

The smart-home lighting controller adjusts brightness using ambient light sensors. The program contains switch-case statements and complex Boolean expressions. Relying solely on statement coverage may leave logical errors undetected, potentially causing incorrect brightness adjustments or hardware-control failures.

2. Analysis of Statement Coverage Limitations

Statement coverage verifies whether executable statements have been executed at least once. However, it does not guarantee that every branch or individual Boolean condition has been tested.

For example, a lighting control statement may execute when the ambient light is low, while the alternative behavior for high ambient light remains untested.

3. Illustrative Control Logic

Assume the controller uses the following conditions:

- (A): Ambient light is below the configured threshold.
- (B): A person is detected in the room.
- (C): The lighting system is operating normally.

The decision to increase brightness is:

[D=(A and B) and C]

This is an illustrative expression used to explain coverage techniques.

4. Branch and Condition Coverage

- Branch coverage: Tests every decision outcome, such as true and false.
- Condition coverage: Ensures that every individual Boolean condition evaluates to both true and false.
- Combined branch and condition coverage: Checks both decision outcomes and individual condition outcomes.

Consider the following test cases:

Test	A: Low Light	B: Person Detected	C: System Normal	Expected Decision
TC1	True	True	True	Increase brightness
TC2	False	True	True	Do not increase brightness
TC3	True	False	True	Do not increase brightness
TC4	True	True	False	Do not increase brightness
TC5	False	False	False	Do not increase brightness

These cases exercise the individual conditions and decision outcomes for the illustrative expression.

5. Testing Switch-Case Statements

Each defined switch-case branch should be tested, including the default branch where applicable.

Test Case	Input Condition	Expected Result
TC1	Very low ambient light	Apply the specified brightness increase
TC2	Normal ambient light	Maintain or apply normal brightness
TC3	High ambient light	Reduce brightness as specified
TC4	Sensor reading unavailable	Apply safe fallback behavior
TC5	Unsupported sensor value	Handle the value without crashing

6. Improved White-Box Testing Strategy

1. Construct a control flow graph from the lighting controller code.
2. Identify all switch-case branches and Boolean expressions.
3. Apply branch and condition coverage.
4. Test threshold boundaries for ambient light.
5. Test sensor failures and invalid readings.
6. Verify the output commands sent to the lighting hardware.
7. Perform integration testing with actual or simulated sensors and lighting devices.
8. Repeat regression tests after modifying the control logic.

7. Justification

Branch coverage identifies untested decision outcomes, while condition coverage examines individual Boolean conditions. Testing switch-case alternatives and hardware interactions further reduces the risk of defects that may not appear during ordinary software execution.

Conclusion: A white-box testing strategy combining statement, branch, and condition coverage provides stronger evidence of correct lighting logic. Integration testing is also necessary to verify that the software sends the correct commands to the hardware.

QUESTION 14: Logistics Management System – Invalid Shipping Weights

Answer:

1. Introduction

The logistics management system calculates shipping costs using package weight, destination zone, and delivery speed. Errors were not detected when zero or negative weights were entered. This indicates that the existing input-validation tests focused primarily on valid values and did not sufficiently examine invalid boundaries.

2. Analysis of the Existing Testing Approach

The current testing strategy may have used only typical positive weights. Consequently, it may not have identified defects such as:

- Accepting zero weight when it is not permitted.
- Accepting negative package weights.
- Incorrectly calculating shipping costs for invalid inputs.
- Failing to display validation messages.
- Allowing invalid records into the shipping database.

Negative weights are not physically meaningful for ordinary shipping calculations, so they must be rejected according to the specified requirements.

3. Application of Robust Boundary Value Analysis

Assume the permitted package-weight range is greater than 0 kg and up to 100 kg, inclusive. These are illustrative limits.

Test Case	Weight	Expected Result
TC1	−0.01 kg	Reject
TC2	0 kg	Reject
TC3	0.01 kg	Accept
TC4	0.02 kg	Accept
TC5	99.99 kg	Accept
TC6	100 kg	Accept

TC7	100.01 kg	Reject

If the actual system permits zero-weight documents or has different weight limits, the test cases must be adjusted to its requirements.

4. Equivalence Partitioning

Equivalence Class	Example Input	Expected Result
Negative weight	−5 kg	Reject
Zero weight	0 kg	Reject under the assumed policy
Valid weight	5 kg	Accept
Above maximum	101 kg	Reject
Non-numeric weight	ABC	Display validation error
Missing weight	Blank	Require a valid value

5. Additional Input Combinations

Weight validation must also be tested with destination zones and delivery speeds.

Weight	Destination Zone	Delivery Speed	Expected Result
Valid	Valid	Standard	Calculate the correct cost
Valid	Valid	Express	Apply express pricing
Invalid negative weight	Valid	Standard	Reject shipment
Valid	Invalid	Standard	Reject invalid destination
Valid	Valid	Unsupported option	Reject or request a valid option

6. Improved Testing Strategy

1. Confirm permitted weight limits and precision.
2. Apply Robust Boundary Value Analysis to valid and invalid boundaries.
3. Use ECP to represent valid, invalid, missing, and non-numeric inputs.
4. Test combinations of weight, zone, and delivery speed.
5. Verify shipping charges against independently calculated expected results.
6. Confirm that invalid shipments are not stored or dispatched.
7. Perform regression testing after fixing validation defects.

7. Justification

Robust Boundary Value Analysis includes invalid values below and above the permitted range. This is particularly important for shipping calculations because accepting invalid inputs may cause incorrect charges, failed shipments, or corrupted records.

Conclusion: Robust BVA and ECP provide systematic coverage of valid and invalid shipping weights. They improve input validation and help ensure accurate shipping-cost calculations.

QUESTION 15: Medical Imaging Software – Prime-Number Dataset Failures

Answer:

1. Introduction

The medical imaging software processes large datasets using parallel-processing loops. Inconsistent results occur when the dataset size is a prime number. The team's reliance on random testing is insufficient because the defect appears to depend on a specific mathematical property of the input size.

A structured white-box approach using Basis Path Testing, loop analysis, and numerical validation is required.

2. Analysis of the Existing Random Testing Strategy

Random testing may generate many dataset sizes without consistently targeting prime numbers. As a result, the specific condition causing the failure may remain undetected.

Potential causes include:

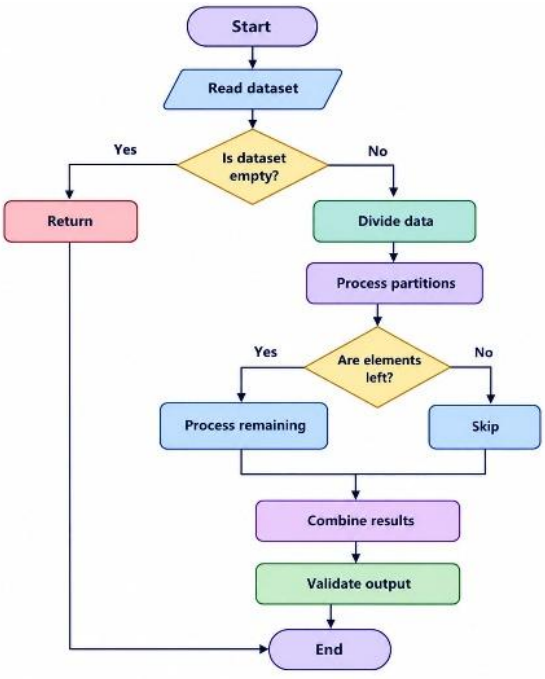
- Incorrect division of data among processing threads.
- Improper handling of leftover elements.
- Incorrect loop termination conditions.
- Array-indexing errors.
- Race conditions during parallel execution.
- Incorrect synchronization or result aggregation.

These are possible causes that must be investigated rather than assumed to be the confirmed defect.

3. Application of Basis Path Testing

Basis Path Testing identifies independent execution paths in the program's control flow graph.

Consider the following illustrative pseudocode:



The graph represents dataset validation, partition processing, handling of remaining elements, and result aggregation.

4. Proposed Test Cases

Test Case	Dataset Size	Purpose	Expected Result
TC1	0	Empty dataset	Handle according to specification
TC2	1	Minimum non-empty dataset	Produce correct result
TC3	2	Small prime size	Produce correct result
TC4	3	Prime size	Produce correct result
TC5	4	Non-prime size	Produce correct result
TC6	7	Prime size	Produce correct result
TC7	8	Non-prime size	Produce correct result
TC8	101	Larger prime size	Produce correct result
TC9	100	Nearby non-prime size	Produce correct result
TC10	Large dataset with leftover elements	Test partition boundaries	Process every element exactly as required

These cases help investigate whether the problem is related to prime sizes, dataset size, partition remainders, or other input characteristics.

5. Control Flow and Cyclomatic Complexity

Cyclomatic complexity estimates the number of independent paths in a control flow graph.

[V(G)=E-N+2]

where (E) is the number of edges and (N) is the number of nodes in a connected graph.

The testing team should calculate the complexity of the actual implementation and select a basis set of independent paths. Every selected path should have a test that exercises its distinct control-flow behavior.

6. Parallel-Processing and Mathematical Validation

Basis Path Testing alone is not sufficient to diagnose every parallel-processing defect. Additional tests should include:

1. Compare parallel results with a trusted sequential implementation.
2. Test prime and non-prime dataset sizes.
3. Test dataset sizes that produce different partition remainders.
4. Verify that every input element is processed exactly as intended.
5. Repeat tests to identify nondeterministic output.
6. Examine synchronization, shared variables, and race conditions.
7. Check numerical precision and acceptable error tolerances.

8. Test large datasets for memory and performance problems.

7. Improved White-Box Testing Strategy

1. Construct the actual control flow graph.
2. Calculate cyclomatic complexity.
3. Identify independent execution paths.
4. Perform Basis Path Testing.
5. Apply loop-boundary testing.
6. Test prime and non-prime dataset sizes systematically.
7. Compare parallel and sequential outputs.
8. Perform regression testing after correcting the defect.

8. Justification

Random testing is useful for discovering unexpected defects, but it may not systematically cover specialized mathematical input conditions. Basis Path Testing provides structured control-flow coverage, while targeted prime-size tests and parallel-versus-sequential comparisons help identify partitioning and numerical errors.

Conclusion: Combining Basis Path Testing, cyclomatic complexity analysis, loop-boundary testing, and targeted numerical validation provides a more reliable strategy for diagnosing inconsistent medical imaging results. This approach improves mathematical correctness and system stability.

Code of Conduct:

I R.RACHITHA/192421058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification